

cash-flow-app - Project Context

Project Overview

Cash Flow App is an Angular 19+ financial management application designed to help users track and manage their monthly cash flow, investments, and various types of installment payments (manual, loan-based, and milestone-based).

Key Purpose: Provide comprehensive visibility into financial commitments, payment schedules, and investment tracking.

Tech Stack

Frontend

- **Framework:** Angular 19.0.0 (standalone components)
- **Styling:** SCSS + Bootstrap 5.3.8 + Angular Material 19.2.19
- **HTTP Client:** Angular HttpClient with custom interceptors
- **State Management:** RxJS BehaviorSubject (no NgRx)
- **i18n:** @ngx-translate/core 17.0.0 (supports Hebrew + English)
- **PDF Export:** jspdf 4.2.1 + jspdf-autotable 5.0.7

Backend

- Node.js server (Express assumed, running on separate port)
- API endpoints for CRUD operations on installments, investments, cash flow

Development

- Angular CLI 19.0.2
- TypeScript 5.6.2
- Jasmine/Karma for testing
- Concurrently for running frontend + backend + optional Ollama simultaneously

Notable Dependencies

- `@angular/cdk` - Component DevKit for overlays, virtual scrolling
- `@angular/animations` - Animation utilities
- `@angular/material` - Pre-built Azure-Blue theme
- `zone.js` - Angular zone management

- `tslib` - TypeScript utilities

Project Structure

```
src/
├── app/
│   ├── components/
│   │   ├── installments/           # Main installments feature
│   │   ├── installments-table/     # Table view (reusable component)
│   │   ├── installment-card/       # Card view (reusable component)
│   │   ├── installment-form/       # Add/edit dialog
│   │   └── installment-confirm-dialogs/ # Delete/payment confirmation
│   └── modals
│       ├── installments-header/     # Header with actions
│       ├── installments-empty-state/ # Placeholder when no data
│       ├── cash-flow-table/         # Monthly cash flow tracking
│       ├── investment-dashboard/     # Investment tracking
│       ├── investment-list/         # Investment list view
│       ├── investment-detail/       # Investment detail modal
│       ├── investment-simulation/    # What-if scenarios
│       └── ai-assistant/             # AI-powered advice (with Ollama
integration)
│   │   └── app.component.ts         # Root component with tabs
│   │
│   └── services/
│       └── installment.service.ts   # Installment CRUD + business
logic
│   │   ├── cash-flow.service.ts    # Monthly cash flow calculations
│   │   ├── investment.service.ts    # Investment CRUD + analysis
│   │   ├── ai.service.ts           # Ollama/LLM integration
│   │   ├── ai-report.service.ts     # Financial report generation
│   │   ├── conversation.service.ts  # Chat history management
│   │   ├── language.service.ts      # i18n management (Hebrew/English)
│   │   └── theme.service.ts         # Dark/light mode
│   │
│   └── models/
│       └── installment.model.ts     # Installment, Loan, Milestone
interfaces
│   │   └── investment.model.ts      # Investment interfaces
│   │
│   └── interceptors/
```

```
| | └─ http.interceptor.ts          # Global HTTP error handling
| |
| └─ app.config.ts                 # Provider configuration
| └─ app.routes.ts                 # Routing config
| └─ app.component.ts             # Root component
|
└─ environments/
  └─ environment.ts                # Development config
  └─ environment.prod.ts           # Production config
  |
  └─ assets/
    └─ i18n/                       # Translation JSON files
      └─ en.json
      └─ he.json
  |
  └─ styles.scss                   # Global styles
  └─ main.ts                       # App bootstrap
  └─ index.html                    # HTML entry point

public/                            # Static assets
server/                            # Node backend (not in src/)
  └─ index.js
```

Core Domain Models

Installment

Main financial commitment entity. Can be one of three types:

```
typescript
```

```

interface Installment {
  id: string;
  name: string;
  totalAmount: number;           // Total sum to pay
  downPayment: number;           // Upfront payment
  monthlyPayment: number;        // Monthly/regular installment
  installmentsCount: number;      // Max number of payments
  startDate: string;             // YYYY-MM format
  color: string;                 // Card color (hex)
  notes: string;
  manualPaidCount: number;        // For manual payment tracking
  lastManualPaymentDate?: string;
  payments?: Array<{date, amount}>;
  paymentType: 'manual' | 'loan' | 'milestone'; // TYPE OF PAYMENT
  loanComponents: LoanComponent[];
  milestones?: Milestone[];
  milestonePayments?: MilestonePayment[];
}

```

LoanComponent

Represents a loan tied to an installment:

- Separate entities with their own amortization schedules
- Each has monthly payments, start/payoff dates, interest rates
- Can be marked paid individually within the parent installment

Milestone

Percentage-based payment points (e.g., 25% down, 50% midway, etc.):

- Defined as percentage + description
- Computed amount based on parent installment total
- Can be marked paid independently

InstallmentStatus

Computed view of an installment with aggregated data:

- `progressPct`: Overall completion percentage
- `paidAmount`: Total paid so far
- `remainingAmount`: Still owed
- `monthsLeft`: Estimated time to payoff

- `isCompleted`: Boolean flag
 - `loanStatuses`: Array of computed loan states
 - `combinedPaymentHistory`: Merged history from all payment types
 - `upcomingPayments`: Forecast of next due dates
-

Key Features

1. Installment Management

- **Create:** Add new installments with type selection (manual/loan/milestone)
- **Edit:** Modify installment details
- **Delete:** Remove with confirmation dialog
- **Track:** View payment progress, history, and remaining balance

2. Payment Tracking

- **Manual Payments:** Fixed monthly amounts tracked directly
- **Loan Payments:** Amortized loans with interest, monthly tracking
- **Milestone Payments:** Percentage-based checkpoints for project-based spending (e.g., construction)
- **Undo/Correct:** Reverse last payment with single click

3. Cash Flow Analysis

- **Monthly View:** Aggregate all scheduled payments by month
- **Cash Flow Warnings:** Flag months with potential shortfalls
- **Combined History:** Unified payment history across all installment types

4. Investment Tracking

- Separate module for portfolio management
- What-if scenario simulation
- Export capabilities

5. AI Assistant (Optional)

- Ollama integration for local LLM
- Financial insights and recommendations
- Conversation history persistence

6. Internationalization

- Full Hebrew + English support
 - Toggle via language service
 - Translation files in `assets/i18n/`
-

Component Architecture

Installments Module

```
InstallmentsComponent (parent, handles logic)
├─ InstallmentsHeaderComponent (title, actions)
├─ InstallmentsTableComponent (list view - reusable)
│   └─ InstallmentCardComponent × N (individual cards)
└─ InstallmentsEmptyStateComponent (no data state)
```

Related Modals:

```
├─ InstallmentFormComponent (create/edit)
├─ InstallmentConfirmDialogsComponent (delete, mark paid)
└─ PaymentActionPayload interface (event shape)
```

Key Implementation Details

Shared Event Types

```
typescript

export interface PaymentActionPayload {
  item: Installment;
  loan?: LoanComponent;
  loanId?: string;
  milestoneId?: string;
}

export interface MilestoneActionPayload {
  item: Installment;
  milestoneId: string;
}
```

Important: These payloads are defined in `installments-table.component.ts` and imported by `installment-card.component.ts`.

InstallmentCardComponent

- **Input:** `(item)` (Installment), `(status)` (InstallmentStatus), `(expandedLoans)` (expand/collapse state)
- **Outputs:** Payment events, edit/delete, history toggle
- **Performance:** Uses `(OnChanges)` to pre-compute `(Set<string>)` of paid milestone IDs (O(1) lookup vs `Array.some()` every CD cycle)

InstallmentCardComponent HTML Template Notes

- Uses **Angular control flow** (`(@if)`, `(@for)`) — Angular 17+ syntax
 - `(st)` should be renamed to `(status)` for clarity (template uses `(st.)`)
 - Milestone paid state checked via `(isMilestonePaid(milestoneId))` method
 - Payment history shown in expandable popups under cards
-

Service Layer

InstallmentService

- **Load:** Fetch all installments from API
- **Add/Update/Delete:** CRUD operations
- **markAsPaid:** Record payment, update `(paidCount)`, append to `(payments[])`
- **markMilestoneAsPaid:** Create `(MilestonePayment)` record, append to `(milestonePayments[])`
- **undoPayment:** Reverse last payment (implementation needed)
- **getStatuses:** Compute `(InstallmentStatus[])` with calculated fields

CashFlowService

- **computeMonthlyFlow:** Aggregate all scheduled payments by month
- **detectWarnings:** Identify months with potential cash shortfalls
- **getUpcomingPayments:** Forecast next N months of payments

Other Services

- **LanguageService:** Language toggle, translation loading
- **ThemeService:** Dark/light mode
- **AIService:** Ollama HTTP calls
- **ConversationService:** Chat history persistence

API Contract (Expected)

Installments Endpoints

GET	/api/installments	→ Installment[]
POST	/api/installments	→ Installment
PUT	/api/installments/:id	→ Installment
DELETE	/api/installments/:id	→ void

Payload Notes:

- `PUT` expects full `Installment` object (not Partial)
- `payments`, `milestonePayments` appended on client before send
- Server should return fully updated object

Known Code Patterns

State Management

- `BehaviorSubject<T>` for reactive data
- Services expose `items$` observable + `.value` getter
- Components subscribe via `async` pipe in templates

HTTP Calls

- All wrapped in `tap()` to update local `BehaviorSubject`
- No error handling at service level (delegated to interceptor)
- No request cancellation (potential memory leak on heavy navigation)

Event Emission

- Parent listens to child `@Output()` events
- Payloads are strongly typed interfaces
- No multi-select or batching (single item operations only)

Build & Run

Development

```
bash

npm install
npm start           # Runs on http://localhost:4300
npm run dev         # Frontend + backend concurrently
npm run dev:ai      # Frontend + backend + Ollama (Windows only path hardcoded)
```

Production

```
bash

npm run build
# Output: dist/cash-flow-app/
```

Important Notes

i18n

- Translation files at `src/assets/i18n/{en,he}.json`
- All UI strings should use `{{ 'KEY' | translate }}`
- Hebrew text appears in templates but should be extracted to JSON

Styling

- SCSS global scope in `src/styles.scss`
- Component-level styles in `.component.scss` files
- Bootstrap 5 utilities available (`.d-flex`, `.mt-2`, etc.)
- Material Design tokens used (colors, spacing)

RTL (Right-to-Left) for Hebrew

- **Not currently implemented** — would require:
 - `dir="rtl"` in HTML
 - RTL-aware SCSS (margins/padding swaps)
 - Material RTL module

Performance Considerations

- No virtual scrolling yet (OK for <500 items)
- No lazy loading of modules (all imported eagerly)
- Installment card `isMilestonePaid()` runs on every change detection if not optimized

Testing

- Jasmine/Karma configured but minimal coverage
 - No e2e tests present
-

Future Improvements / Tech Debt

1. **RTL Support:** Enable full Hebrew right-to-left layout
 2. **State Management:** Consider NgRx for complex state
 3. **Performance:** Virtual scrolling for large lists, lazy modules
 4. **Error Handling:** Consistent error UI across all services
 5. **Testing:** Expand unit/e2e test coverage
 6. **Accessibility:** ARIA labels, keyboard navigation audits
 7. **Backend Integration:** Standardize API error responses, add pagination
 8. **AI Integration:** Move Ollama endpoint to environment config
-

Development Workflows

Adding a New Installment Type

1. Define interface in `installment.model.ts`
2. Extend `Installment` discriminated union with new `paymentType`
3. Add service logic in `installment.service.ts` for the new type
4. Extend `InstallmentCardComponent` to handle rendering + actions
5. Add translation keys to `en.json` and `he.json`

Debugging

- Browser DevTools Console — check `window.installments` (if exposed)
- Network tab — inspect API calls and payloads
- Material's debug overlay (if enabled) for layout issues

- RxJS subscription tracking (check for memory leaks)
-

Environment Configuration

Development:

- API URL typically `http://localhost:3000` or similar
- Set in `environment.ts`

Production:

- API URL must be updated in `environment.prod.ts`
- Build with `npm run build`

Example `environment.ts`:

```
typescript

export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api'
};
```

Useful Commands

```
bash
```

```
# Check TypeScript errors
ng build

# Serve with source maps
ng serve --source-map

# Build for production
ng build --configuration production

# Run tests
ng test

# Lint (if configured)
ng lint
```

Contact & Contribution

- **Language:** Primarily Hebrew in UI, English in code
 - **Code Style:** Consistent with Angular style guide
 - **PR Requirements:** Maintain TypeScript strict mode, add translation keys for UI strings
-

Document Last Updated: June 2026 **Angular Version:** 19.0.0 **Status:** Active Development